

Architecting Multi-Agent Swarms with the Cursor SDK: Orchestration Frameworks, Coordination Topologies, and Human-in-the-Loop Governance

Modern software engineering systems have evolved beyond conversational code assistants, establishing autonomous coding agents as programmable infrastructure¹. The programmatic agent interface exposed by Cursor through its TypeScript (@cursor/sdk) and Python SDKs abstracts the harness that powers the Cursor desktop application, the command-line interface, and sandboxed cloud environments⁴. This harness packages codebase indexing, semantic search, fast lexical search, and automated file reconciliation into a scriptable runtime². Constructing a collaborative swarm of software agents on top of this runtime requires resolving core distributed systems challenges: non-deterministic execution drift, context contamination, race conditions across shared workspaces, and unconstrained file mutations³. Solving these operational challenges demands an architecture that separates high-level workflow coordination from low-level workspace actions⁷. By combining stateful graph orchestration with the Cursor SDK and enforcing multi-tiered human-in-the-loop (HITL) checkpoints, engineering teams can build resilient agent swarms capable of executing complex engineering objectives safely and autonomously⁸.

1. The Cursor SDK Architecture and Execution Abstractions

The Cursor SDK acts as an operational agent harness rather than a raw model inference or chat completion endpoint⁴. When invoked, the SDK initializes an agent runtime equipped with the workspace tools, codebase traversal capabilities, and tool-dispatch loops found in the Cursor editor³. This design offloads the undifferentiated heavy lifting of context management, diff reconciliation, and prompt assembly to the native Cursor execution loop².

Core Runtime Primitives

The SDK structures agent workflows around three fundamental primitives: the Agent, the Run, and normalized stream messages⁴.

The Agent primitive serves as a durable, stateful handle that encapsulates conversation history, workspace bindings, tool configurations, and underlying model selections across multiple interactions⁴. Rather than rebuilding context on every request, the Agent retains its conversational state while managing file system attachments⁴.

A Run represents a single prompt submission dispatched to an active Agent instance⁴. Each Run owns its lifecycle, tracking real-time status, token consumption, cancellation handles, and streaming response events⁴. Cloud-hosted agents enforce a single active Run per Agent at any

given time, ensuring deterministic progression through a task¹¹. Normalized stream messages (SDKMessage) provide typed, real-time events emitted throughout an active Run⁴. These messages represent a discriminated union across assistant output tokens, internal reasoning traces, tool execution arguments, execution errors, and lifecycle status changes⁴. Because the stream message schema is identical across all execution targets, orchestrators can monitor local and remote agents using uniform ingestion pipelines⁴.

Execution Environments: Local Workspaces Versus Sandboxed Cloud VMs

The Cursor SDK abstracts two primary runtime environments through a unified API interface, determined by whether configuration parameters specify a local workspace directory or cloud repository configurations⁴.

```
TypeScript
import Agent from "@cursor/sdk"

// 1. Local Runtime: Mounts directly to the host machine working tree
const localAgent = await Agent.create({
  apiKey: process.env.CURSOR_API_KEY,
  model: id "composer-2.5",
  local: cwd process.cwd()
})

// 2. Cloud Runtime: Provisions a remote, isolated VM in Cursor's infrastructure
const cloudAgent = await Agent.create({
  apiKey: process.env.CURSOR_API_KEY,
  model: id "composer-2.5",
  cloud: {
    repos: { url "https://github.com/org/platform-core" startingRef "main" },
    autoCreatePR: true
  }
})
```

The operational characteristics of these environments dictate their role within an agent swarm:

Operational Parameter	Local Runtime (local)	Cloud Runtime (cloud)
Execution Sandbox	Directly touches the host	Ephemeral, isolated virtual

	file system (cwd) ³	machine ²
Session Persistence	Process-bound; halts if the local script terminates ⁴	Durable; runs asynchronously in Cursor cloud infrastructure ²
Version Control Integration	Directly mutates working tree files ³	Automates branches, commits, and Pull Request creation ²
Artifact Generation	Local file modifications only ³	Screenshots, browser interaction traces, and execution videos ³
Identifier Scheme	agent-* prefix ⁴	bc-* prefix (linked to Web UI and IDE Agents Window) ²
Concurrency Ceiling	Constrained by local disk access and process resources ³	Horizontally scalable across parallel cloud instances ³

Because a single Agent instance operates sequentially on its designated workspace, creating a concurrent swarm requires managing a fleet of distinct Agent instances rather than running parallel calls through a single agent session³.

2. Multi-Agent Swarm Topologies

Naive agent swarms that rely on unstructured peer-to-peer communication—where all agents post messages to a shared context pool—suffer from rapid token exhaustion, conversational loops, and coordination failures³. Production systems avoid these issues by using structured coordination topologies that isolate context and enforce clear lines of responsibility⁷.

The Hierarchical Planner-Worker Pattern

The hierarchical planner-worker pattern organizes agents into distinct tiers of abstraction⁷. A root planner agent decomposes high-level engineering objectives into discrete work packages but is explicitly barred from editing code directly⁷. This separation keeps the planning context clean and focused on overall system architecture⁷.

Once the planning agent generates an execution plan, an orchestration script distributes these tasks to dedicated worker agents⁷. Crucially, worker agents operate in separate workspaces or cloud VMs and do not communicate directly with one another⁷. Instead, workers report their results upward through structured handoff summaries⁷. These summaries detail modified files, test run outputs, remaining work, and execution blockers⁷. The planner ingests these structured handoffs to track progress, resolve dependencies, and dynamically adjust downstream tasks⁷.

Topological Graph Execution via DAG Task Runners

Workflows with deterministic dependency chains—such as migrating database schemas, generating backend endpoints, and updating client SDKs—are best modeled as Directed Acyclic Graphs (DAGs)¹⁴. In this setup, each node in the graph represents a dedicated Cursor agent assigned to a single scoped task³.

A centralized orchestration engine tracks dependency edges across the graph³. As upstream dependencies complete and validate their changes, the engine identifies unblocked downstream tasks and schedules worker agents to run in parallel⁷. This approach prevents race conditions by ensuring multiple agents never attempt to edit the same files at the same time³.

Context-Isolated Subagents

The Cursor SDK supports subagent delegation through configuration options passed to `Agent.create()` or via markdown templates stored in `.cursor/agents/*.md`³. Subagents serve primarily as context isolation boundaries rather than engines of massive parallelism³.

When a parent agent delegates a subtask (such as deep codebase search, running test suites, or drafting documentation) to a named subagent, the parent pauses its own execution loop until the subagent finishes³. The subagent runs inside its own isolated context window, discarding intermediate chatter and returning only its final result to the parent³. This keeps noisy search logs, verbose compile errors, and test output from polluting the parent agent's reasoning context³.

3. Technology Stack Evaluation for Cursor Swarms

Selecting the optimal orchestration framework to manage a Cursor SDK swarm requires evaluating state management, error recovery, scheduling capabilities, and tool interoperability⁸.

Framework / Engine	Core Abstraction	Concurrency Model	Human-in-the-Loop Architecture	Cursor SDK Integration Effort	Recommended Use Case
LangGraph (TypeScript / Python)	Stateful directed graphs with cycles and conditional branching ⁸	Deterministic, branch-based parallel node scheduling ⁸	Built-in <code>interrupt()</code> mechanism backed by durable state checkpoints ⁸	Low; wraps SDK calls cleanly inside graph nodes ⁴	Complex enterprise software delivery; pipelines with feedback loops ⁸
Native DAG	Topological task graphs	Asynchronous worker	Stdio interceptor	Zero; built natively	Self-contained

Engine (Cookbook Pattern)	executed over JSON manifests ¹⁴	queues utilizing native concurrency primitives ³	scripts and external CLI approval gates ³	with TypeScript and SDK primitives ¹⁴	developer tooling, repository refactoring, and local CLI tools ³
Temporal / Inngest	Code-as-configuration durable execution workflows	Distributed worker pools and durable task queues	Long-polling timers, external webhooks, and manual approval signals	Medium; requires wrapping SDK execution loops in durable activity workers	Large-scale enterprise swarms operating across multiple repositories ³
AutoGen / OpenAI Agents SDK	Conversational peer groups and conversational handoffs ¹⁶	Event-driven conversational messaging loops ⁸	Text-based prompt interruptions and user proxy agents ¹⁷	High; chat-based abstractions conflict with file-centric workspaces ⁴	Open-ended exploratory tasks, multi-agent debates, and creative ideation ¹⁶

The Winning Combination: LangGraph and the Cursor SDK

For engineering swarms that demand rigorous structure, **LangGraph paired with the Cursor SDK** provides the most resilient architecture⁸.

LangGraph acts as the overarching cognitive and scheduling layer: it maintains global swarm state, evaluates conditional routing paths, handles execution rollbacks, and safely suspends workflows when waiting for human input⁸. Meanwhile, the Cursor SDK serves as the physical execution harness: it performs repository indexing, resolves workspace dependencies, uses semantic grep, executes terminal commands, and reconciles file changes⁴.

TypeScript

```
import StateGraph, Annotation from "@langchain/langgraph"
import Agent from "@cursor/sdk"
```

```

// Define the collective multi-agent workspace state
const SwarmState = Annotation.Root({
  taskDescription: Annotation,
  architectureSpec: Annotation,
  implementationFiles: Annotation,
  verificationStatus: Annotation<"pending" | "approved" | "rejected">,
  humanFeedback: Annotation,
});

// Implementation node executing through the Cursor SDK
async function implementationNode(state: typeof SwarmState.State) {
  const agent = await Agent.create({
    apiKey: process.env.CURSOR_API_KEY!,
    model: { id: "composer-2.5" },
    local: { cwd: process.cwd() },
  });

  const prompt = `Implement the following architecture specification:
${state.architectureSpec}
Focus strictly on writing implementation and unit tests.`;

  const run = await agent.send(prompt);
  await run.wait();

  return { implementationFiles: ["src/feature.ts", "test/feature.test.ts"] };
}

```

The Model Context Protocol (MCP) as a Universal Tool Layer

The Model Context Protocol (MCP) establishes an open standard for connecting agents to external systems, replacing bespoke integration code with unified tool servers². The Cursor SDK natively supports MCP, allowing developers to configure servers globally via `.cursor/mcp.json` or register them dynamically at runtime using `Agent.create({ mcpServers: { ... } })`⁴.

By standardizing on MCP, every agent in the swarm can safely interact with database instances, internal issue trackers (e.g., Jira, Linear), and observability platforms⁶. This design decouples tool definitions from the core orchestration logic, ensuring that tools can be added, updated, or audited without modifying the underlying agent workflows².

4. Human-in-the-Loop (HITL) Governance and Steering Mechanisms

Granting autonomous swarms permission to edit code, execute terminal commands, and

manage branches introduces serious operational risks, including accidental data loss, security vulnerabilities, and runaway compute costs³. To mitigate these hazards, production systems implement a defense-in-depth safety model spanning three distinct layers: lifecycle hooks, orchestrator checkpoints, and visual review interfaces⁸.

Layer 1: Deterministic Interception via Cursor Lifecycle Hooks

The Cursor harness includes a lifecycle hook mechanism configured through `.cursor/hooks.json`⁹. Hooks run as independent child processes that communicate with the agent loop using JSON over standard input and output⁹. These hooks execute immediately before and after critical actions, providing fine-grained control over the agent's behavior⁹. Key intercept points include `beforeShellExecution` and `afterShellExecution` for terminal access⁹, `preToolUse` and `postToolUse` for internal tools⁹, `beforeReadFile` and `afterFileEdit` for disk operations⁹, and `subagentStart` and `subagentStop` for child tasks⁹. When an action triggers a hook, the script inspects the input payload and returns a verdict: `allow`, `deny`, or `ask`⁹. Cursor resolves conflicting verdicts across configurations using a strict security precedence: **deny** overrides **ask**, and **ask** overrides **allow**⁹.

```
JSON
{
  "version": 1,
  "hooks": {
    "beforeShellExecution": {
      "command": "/scripts/guardrails/validate-shell.sh",
      "matcher": "rm|npm publish|git push|kubectl",
      "timeout": 15,
      "failClosed": true
    },
    "preToolUse": {
      "command": "/scripts/guardrails/verify-file-access.sh",
      "matcher": "Write|Delete"
    }
  }
}
```

Setting `"failClosed": true` is essential for production security⁹. By default, hook crashes or

timeouts fail open, allowing the action to proceed⁹. Enabling failClosed ensures that if a security hook times out or fails, the requested action is blocked automatically⁹.

```
Bash
#!/usr/bin/env bash
# ./scripts/guardrails/validate-shell.sh
# Stdio hook validating agent shell execution payloads

read -r PAYLOAD
COMMAND=$(echo "$PAYLOAD" | jq -r '.command // empty')

# Prohibit forced Git pushes and destructive cluster commands outright
if [ "$COMMAND" == "git push" || "$COMMAND" == "git push --force" || "$COMMAND" == "kubectl delete" ] then
    echo '{"permission": "deny", "user_message": "Hard-blocked destructive command detected."}'
    exit
fi

# Intercept infrastructure commands and prompt for human approval
if [ "$COMMAND" == "terraform apply" || "$COMMAND" == "aws s3 rm" ] then
    echo '{"permission": "ask", "user_message": "External infrastructure state change requires human sign-off."}'
    exit
fi

echo '{"permission": "allow"}'
exit
```

Layer 2: Dynamic Approval Checkpoints in State Machines

While lifecycle hooks catch low-level command syntax risks, orchestrator checkpoints evaluate high-level business logic and architecture decisions⁸. When using LangGraph, workflows can be configured to pause before critical stages using native `interrupt()` functions⁸.

```
TypeScript
import { interrupt } from "@langchain/langgraph"

async function humanApprovalNode(state: typeof SwarmState.State)
```

```

// Suspend execution, persist graph state, and alert the human operator
const approvalDecision = interrupt()

question "Review modified source files and approve deployment step."
files state.implementationFiles
generatedCodeSummary state.architectureSpec

//

if approvalDecision.verdict !== "APPROVED"
  return
  verificationStatus "rejected"
  humanFeedback approvalDecision.reason
  //

return verificationStatus "approved"

```

When an interrupt() runs, the orchestrator persists the complete execution state to durable storage and notifies the development team via webhooks, Slack, or internal dashboards⁷. The workflow remains paused until an engineer reviews the proposal and submits feedback, allowing the pipeline to safely resume or pivot based on human instruction⁷.

Layer 3: Visual Steering and Workspace Inspection

Supervising multi-agent swarms requires visibility into the changes being made². The Cursor ecosystem provides several mechanisms for inspecting active runs:

When using the Cloud runtime, setting autoCreatePR: true ensures that completed work is isolated on a feature branch, with tests, summaries, and code diffs submitted as a pull request for standard code review². For real-time monitoring, cloud runs emit durable identifiers (bc-*) that stream progress directly into the Cursor IDE Agents Window and web console, allowing engineers to inspect running tasks and step in to take manual control whenever necessary². Additionally, orchestration platforms can stream state changes directly into visual canvases or terminal dashboards, providing live visibility as tasks move through the swarm³.

5. End-to-End Architectural Blueprint

The following blueprint implements a multi-agent software engineering swarm using @cursor/sdk. The system coordinates three specialized roles: a Systems Architect, an Implementation Worker, and an Automated Code Reviewer, linked together by an interactive human approval gate.

TypeScript

```
import { Agent, type SDKMessage } from "@cursor/sdk"
import { readline } from "node:readline/promises"
import { stdin as input, stdout as output } from "node:process"
```

```
interface SwarmManifest {
  task: string
  architecturePlan?: string
  changedFiles: string
  reviewStatus: "PENDING" | "PASSED" | "FAILED"
}
```

```
class AgentSwarmCoordinator {
  private baseDir: string
  private manifest: SwarmManifest
```

```
  constructor(baseDir: string, taskDescription: string) {
    this.baseDir = baseDir
    this.manifest = {}
    this.task = taskDescription
    this.changedFiles = []
    this.reviewStatus = "PENDING"
  }
}
```

```
// Phase 1: Planning and Architecture Analysis
async runArchitect(): Promise<void> {
  console.log("\n[1/3] Launching Systems Architect Agent...")
```

```
  // Architect uses intelligent reasoning to parse repository structure
  const architect = await Agent.create({
    apiKey: process.env.CURSOR_API_KEY!,
    model: { id: "composer-2.5" },
    local: { cwd: this.baseDir },
  })
```

```
  try {
    const prompt = `Analyze our codebase and output a technical implementation plan for:
"${this.manifest.task}".
Document exact file paths to create or modify, necessary functions, and test cases.`
```

```
    const run = await architect.send(prompt)
    let planOutput = ""
```

```

    for await (const message of run.stream())
    if (message.type === "assistant")
    for (const block of message.message.content)
    if (block.type === "text")
        planOutput += block.text;

    // ...

    await run.wait();
    this.manifest.architecturePlan = planOutput;
    console.log("[1/3] Technical architecture blueprint finalized.");
    finally
    await architect.Symbol.asyncDispose();
    // ...
    // ...

```

```

// Phase 2: Human Sign-off and Interactive Review
async requestHumanApproval(): Promise<boolean> {
    console.log("\n=====");
    console.log("          HUMAN REVIEW CHECKPOINT          ");
    console.log("=====");
    console.log("Planned Architecture Implementation:\n");
    console.log(this.manifest.architecturePlan?.slice(0, 500) + "...");
}

```

```

const rl = readline.createInterface({ input, output });
const decision = await rl.question("Approve architectural plan? (yes/no): ");
rl.close();

```

```

return decision.trim().toLowerCase() === "yes";
}

```

```

// Phase 3: Isolated Code Implementation
async runWorker(): Promise<void> {
    console.log("\n[2/3] Spawning Implementation Worker Agent...");
}

```

```

// Worker implements code changes guided by the approved architecture plan
const worker = await Agent.create({
    apiKey: process.env.CURSOR_API_KEY!,
    model: { id: "composer-2.5" },
    local: { cwd: this.baseDir },
});

```

```

    try
        const prompt = `Execute the following approved architectural plan:
${this.manifest.architecturePlan}

Implement all requested changes and construct matching unit tests.`

        const run = await worker.send(prompt)

        for await (const message of run.stream()) {
            if (message.type === "tool_call" && message.tool_call.type === "edit") {
                console.log(`-> Agent modifying file: ${message.tool_call.args.path}`)
                this.manifest.changedFiles.push(message.tool_call.args.path)
            }
        }

        const outcome = await run.wait()
        if (outcome.status !== "completed") {
            throw new Error(`Worker run exited with status: ${outcome.status}`)
        }

        console.log("[2/3] Code implementation complete.")
        finally
            await worker.$dispose()

        // Phase 4: Automated Verification and Verification Loop
        async function runReviewer(): Promise<void> {
            console.log("\n[3/3] Deploying Automated Verification & Review Agent...")

            const reviewer = await Agent.create({
                apiKey: process.env.CURSOR_API_KEY!,
                model: { id: "composer-2.5" },
                local: { cwd: this.baseDir },
            })

            try {
                const prompt = `Inspect these modified files:
${JSON.stringify(this.manifest.changedFiles, null, 2)}

Run existing tests using your shell tool and verify formatting.
Output 'REVIEW_STATUS: PASSED' if valid, or 'REVIEW_STATUS: FAILED' with details.`

```

```

const run = await reviewer.send(prompt)
let reviewResult = ""

for await (const message of run.stream()) {
  if (message.type === "assistant") {
    for (const block of message.content) {
      if (block.type === "text") reviewResult += block.text
    }
  }
}

await run.wait()

if (reviewResult.includes("REVIEW_STATUS: PASSED")) {
  this.manifest.reviewStatus = "PASSED"
  console.log("[3/3] Code verification passed successfully.")
} else {
  this.manifest.reviewStatus = "FAILED"
  console.warn("[3/3] Code review flagged issues:\n" + reviewResult)
}

finally {
  await reviewer[Symbol.asyncDispose]()
}

// Master Swarm Execution Pipeline
async execute() Promise void {
  await this.runArchitect()

  const isApproved = await this.requestHumanApproval()
  if (!isApproved) {
    console.log("Run rejected by human operator. Aborting swarm execution.")
    return
  }

  await this.runWorker()
  await this.runReviewer()

  console.log("\nSwarm execution finished.")
  console.log(`Final Status: ${this.manifest.reviewStatus}`)
  console.log(`Files Updated: ${[...new Set(this.manifest.changedFiles)].join(", ")}`)
}

```

```
// System execution endpoint
const task = "Add structured telemetry metrics to API request middlewares."
const swarm = new AgentSwarm({id: 'api-metrics-swarm', task});
await swarm.execute();
```

6. Strategic Takeaways and Operational Recommendations

Deploying multi-agent engineering swarms requires moving past informal prompting techniques toward structured, observable system design¹. The Cursor SDK provides an execution harness that allows teams to run codebase indexing, file editing, and test execution directly through code⁴. However, relying solely on autonomous LLMs without architectural boundaries creates significant operational risks³.

Engineering organizations should decouple the orchestration plane from the physical execution layer⁷. Frameworks like LangGraph provide the required state persistence, conditional branching, and checkpointing infrastructure needed to manage complex workflows⁸. Meanwhile, delegating actual repository modifications to Cursor SDK agents preserves IDE-level code understanding, syntax validation, and tool integration⁴.

Workspaces must be protected through strict structural isolation³. Unconstrained peer-to-peer agent networks frequently succumb to context contamination, conversational loops, and file-write collisions³. In contrast, hierarchical planner-worker patterns and topological DAG runners maintain clean context boundaries: planning agents output structured task manifests, worker agents operate inside isolated repositories, and execution outcomes flow upward through structured handoff summaries⁷. For large-scale parallel tasks, teams should deploy Cursor Cloud VM instances rather than saturating local environments².

Safety and human oversight must be designed directly into the execution pipeline⁹. Production swarms should employ defense-in-depth governance: deterministic hooks configured with `failClosed: true` in `.cursor/hooks.json` to block dangerous terminal commands and file mutations⁹, durable state machine interrupts to require human review before high-risk operations⁸, and visual inspection channels via Cursor Cloud pull requests and the IDE Agents Window². By combining structured graph coordination, the Cursor execution harness, and robust governance gates, teams can build agent swarms that deliver software reliably and safely at scale².

Works cited

1. Cursor SDK Guide: TypeScript vs Python, Pricing & Limits - Kingy AI, <https://kingy.ai/news/cursor-sdk-review-cursors-coding-agent-becomes-programmable-infrastructure/>
2. Build programmatic agents with the Cursor SDK, <https://cursor.com/blog/typescript-sdk>

3. Cursor SDK reference: what you can build with it - The Deep Feed,
<https://www.thedeeptfeed.ai/posts/2026-05-02-cursor-sdk-builder-reference/>
4. Cursor TypeScript SDK | Cursor Documentation,
<https://cursor.com/docs/sdk/typescript>
5. Cursor Python SDK | Cursor Documentation, <https://cursor.com/docs/sdk/python>
6. how to build Agents with Cursor Agents SDK - Composio,
<https://composio.dev/content/how-to-build-agents-with-cursor-agents-sdk>
7. cursor/orchestrate | Decision Hub, <https://hub.decision.ai/skills/cursor/orchestrate>
8. How to think about agent frameworks - LangChain,
<https://www.langchain.com/blog/how-to-think-about-agent-frameworks>
9. Hooks | Cursor Docs, <https://cursor.com/docs/hooks>
10. Cursor Sdk - AI Skill for Any LLM - TypingMind,
<https://www.typingmind.com/skills/cursor-cursor-sdk>
11. Cursor SDK Tutorial: Run Coding Agents With TypeScript - DataCamp,
<https://www.datacamp.com/tutorial/cursor-sdk>
12. Cursor SDK - Braintrust,
<https://www.braintrust.dev/docs/integrations/agent-frameworks/cursor-sdk>
13. MCP Issues with Cursor SDK - #7 by mohitjain - Bug Reports,
<https://forum.cursor.com/t/mcp-issues-with-cursor-sdk/161629/7>
14. GitHub - cursor/cookbook, <https://github.com/cursor/cookbook>
15. AI Agent Orchestration: Multi-Agent Workflow Guide - Digital Applied,
<https://www.digitalapplied.com/blog/ai-agent-orchestration-workflows-guide>
16. 12 Best Multi-Agent Frameworks for Production in 2026 - AY Automate,
<https://www.ayautomate.com/blog/best-multi-agent-frameworks>
17. 10 official GitHub repos from the teams behind today's AI models,
https://www.reddit.com/r/WebAfterAI/comments/1vq12ah/10_official_github_repos_from_the_teams_behind/
18. Context Engineering - LangChain,
<https://www.langchain.com/blog/context-engineering-for-agents>
19. Cursor Docs — Agent, Rules, MCP, Skills & CLI, <https://cursor.com/docs>
20. Cursor Hooks: scripts that sit on the agent loop - Vikas Tiwari,
<https://tiwarivikas.in/writing/cursor-hooks-agent-loop>
21. Cursor: shell command logging and gating - GitHub Gist,
<https://gist.github.com/alejo4373/ea9bc4dc47c0d13ab64a926b5e44019f>
22. Third Party Hooks | Cursor Docs,
<https://cursor.com/docs/reference/third-party-hooks>
23. Deep Dive into the new Cursor Hooks - Butler's Log - GitButler,
<https://blog.gitbutler.com/cursor-hooks-deep-dive>
24. Create Cursor hooks that run scripts or prompts around agent events,
<https://skills.rest/skill/create-hook-shoshoavi>
25. Auto-Approve Coding Agent Permission Prompts with Jev,
<https://openrouter.ai/docs/projects/docs/cookbook/coding-agents/auto-approve-permission-prompts-with-jev>